

Entry/Exit State Machine: กลไกการเข้าและออกจาก Trade

State machine ควบคุมทุก transition — ไม่มี ad-hoc decision

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

แทนที่จะตัดสินใจ "เข้า/ออก" แบบ ad-hoc ทุกครั้ง ให้ออกแบบ **State Machine** ที่กำหนด state ทั้งหมดและเงื่อนไขการ transition อย่างชัดเจน — ป้องกัน race condition, double-entry, และ logic กระจาย

**NO_TRADE → WATCH → PAPER_GO → MANUAL_GO → DE_RISK → EXIT →
INVALIDATED**

11.1 ทำไมต้องมี State Machine

ในระบบ trading จริง สิ่งที่เกิดขึ้นพร้อมกันมีมาก: ราคาอัปเดตทุก millisecond, สัญญาณ cointegration เปลี่ยน, regime model อัปเดต, manual approval เข้ามา — ถ้าไม่มีโครงสร้างที่ชัดเจน โค้ดจะกลายเป็น chaos

ปัญหาที่พบบ่อยเมื่อไม่มี State Machine

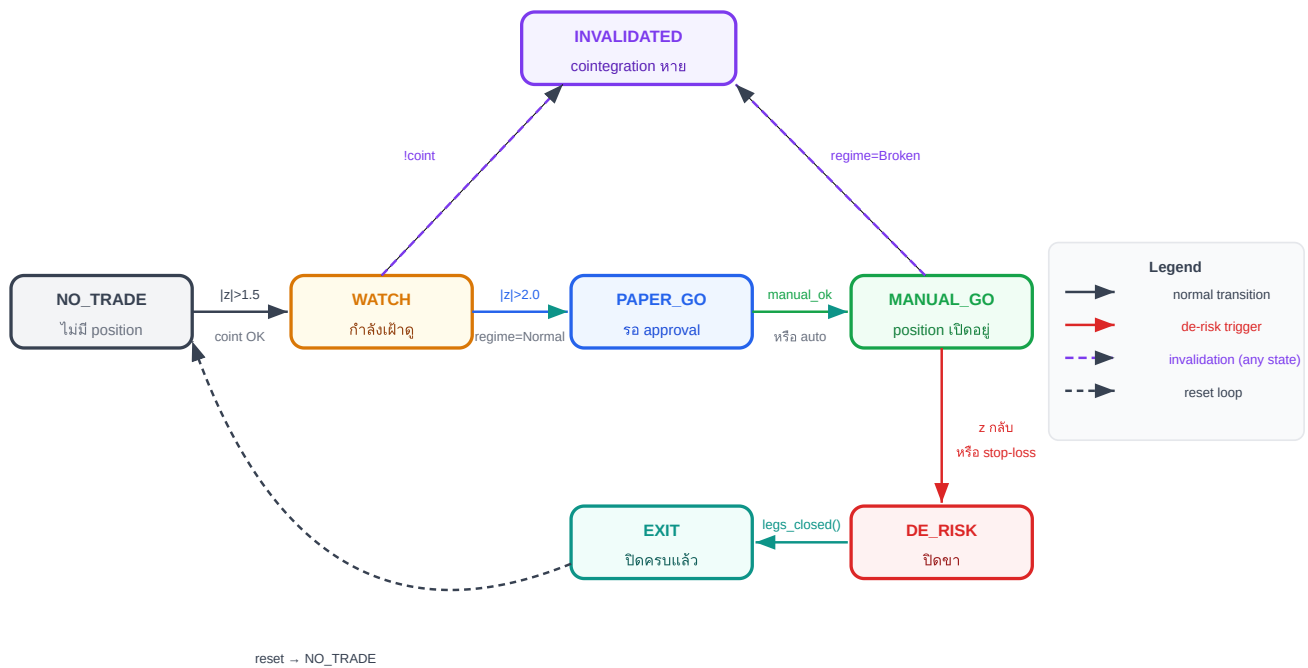
- **Race condition:** เปิด position สองครั้งเพราะ signal มาพร้อมกันสองทาง
- **Double-entry:** ระบบ "ลืม" ว่า position เปิดอยู่แล้ว และเพิ่มขนาดซ้ำ
- **Logic กระจัดกระจาย:** เงื่อนไข if/else กระจายทั่วโค้ด ไม่รู้ว่า "สถานะ" ปัจจุบันคืออะไร
- **Stop-loss ไม่ trigger:** เช็คเงื่อนไขผิด state ทำให้ stop-loss ไม่ทำงาน

ข้อดีของ State Machine

- ทุก transition ต้องผ่านเงื่อนไขที่กำหนดไว้ล่วงหน้า — ไม่มีการข้ามขั้นตอน
- ณ ทุกจังหวะ ระบบรู้ว่าอยู่ใน state ไหน และ action ที่ valid มีอะไรบ้าง
- ง่ายต่อการ log, debug, และ audit trail
- สามารถ test แต่ละ transition แยกกันได้

11.2 State Diagram — 7 States และ Transitions

State machine ของเราประกอบด้วย 7 states หลัก แต่ละ state มี transitions ที่ชัดเจน:



State diagram: ทุก transition มีเงื่อนไขชัดเจน — ไม่มีการข้ามขั้นตอน

11.3 Transition Conditions Table

ตารางด้านล่างสรุป transition ทั้งหมด พร้อมเงื่อนไขและ action ที่เกิดขึ้น:

State ปัจจุบัน	เงื่อนไข	Next State	Action ที่ทำ
NO_TRADE	$ z > 1.5$ AND <code>coint_ok</code> AND <code>regime = Normal</code>	WATCH	บันทึก <code>z_watch</code> , ตั้ง alert
WATCH	$ z > 2.0$ AND <code>coint_ok</code> AND <code>regime = Normal</code>	PAPER_GO	บันทึก <code>entry_zscore</code> , ส่ง paper signal
WATCH	<code>!coint_ok</code> OR <code>regime = Broken</code>	INVALIDATED	ล้าง watch state, log เหตุผล
WATCH	$ z < 1.0$ (signal หาย)	NO_TRADE	reset กลับ
PAPER_GO	<code>manual_ok = True</code> (หรือ <code>auto_mode</code>)	MANUAL_GO	ส่ง order จริงทั้งสองขา
MANUAL_GO	$z \times \text{sign}(\text{entry}_z) < 0.5$ OR <code>stop_loss_hit()</code>	DE_RISK	เริ่มปิด position ที่ละขา
MANUAL_GO	<code>!coint_ok</code> OR <code>regime = Broken</code>	INVALIDATED	ปิด position ทันที (emergency)
DE_RISK	<code>legs_closed() = True</code>	EXIT	บันทึก P&L, ล้าง state
EXIT / INVALIDATED	หลัง cooldown	NO_TRADE	reset ทุก field

11.4 Pseudo-code: StateMachine Class

โค้ดด้านล่างแสดงโครงสร้างหลักของ State Machine — ทุก transition อยู่ใน method เดียว `update()` เพื่อความชัดเจน:

```
class StateMachine:
    def __init__(self):
        self.state = "NO_TRADE"
        self.entry_zscore = None
    def update(self, zscore, regime, coint_ok, manual_ok=False):
        if self.state == "NO_TRADE":
            if abs(zscore) > 1.5 and coint_ok and regime == "Normal":
                self.state = "WATCH"
            elif self.state == "WATCH":
                if abs(zscore) > 2.0 and coint_ok and regime == "Normal":
                    self.state = "PAPER_GO"
                    self.entry_zscore = zscore
                elif not coint_ok or regime == "Broken":
                    self.state = "INVALIDATED"
            elif abs(zscore) < 1.0:
                self.state = "NO_TRADE" # signal หาย reset
        elif self.state == "PAPER_GO":
            if manual_ok:
                self.state = "MANUAL_GO"
            elif self.state == "MANUAL_GO":
                if zscore * sign(self.entry_zscore) < 0.5 or stop_loss_hit():
                    self.state = "DE_RISK"
            elif not coint_ok or regime == "Broken":
                self.state = "INVALIDATED"
        elif self.state == "DE_RISK":
            if legs_closed():
                self.state = "EXIT"
        return self.state
```

หมายเหตุสำคัญเกี่ยวกับโค้ด

- `sign(self.entry_zscore)` บอกทิศทาง: ถ้า entry ที่ $z = +2.3 \rightarrow$ รอให้ z กลับมามากกว่า $+0.5$
- `stop_loss_hit()` ควร implement แยกต่างหาก เช็คทั้ง unrealized P&L และ z-score absolute
- `legs_closed()` เช็คได้ว่าทั้งสองขา (long และ short) ได้รับ fill ครบแล้ว
- ใน production ควรเพิ่ม persistence — บันทึก state ลง DB เพื่อกู้คืนได้หลัง crash

11.5 Running Example: Bybit ↔ Lighter BTC Perp

Running Example: Sequence ของ State Transitions จริง

สมมติเวลา 14:00 น. วันหนึ่ง ระบบเริ่มตรวจพบ signal ใน Bybit↔ Lighter BTC perp pair:

เวลา	z-score	Event	State เก่า	State ใหม่
14:00	0.8	ปกติ ไม่มีสัญญาณ	NO_TRADE	NO_TRADE
14:15	1.7	$ z > 1.5$, coint OK, regime Normal	NO_TRADE	WATCH
14:22	2.3	$ z > 2.0$, confirm signal	WATCH	PAPER_GO
14:25	2.1	trader กด approve (manual_ok=True)	PAPER_GO	MANUAL_GO
14:25	2.1	ส่ง order: Short Bybit, Long Lighter	MANUAL_GO	MANUAL_GO
15:10	0.4	$z < 0.5$ (mean reversion สำเร็จ)	MANUAL_GO	DE_RISK
15:12	0.3	ปิดทั้งสองขาครบ (legs_closed)	DE_RISK	EXIT
15:20	—	หลัง cooldown 8 นาที	EXIT	NO_TRADE

ผลลัพธ์ของ Trade นี้

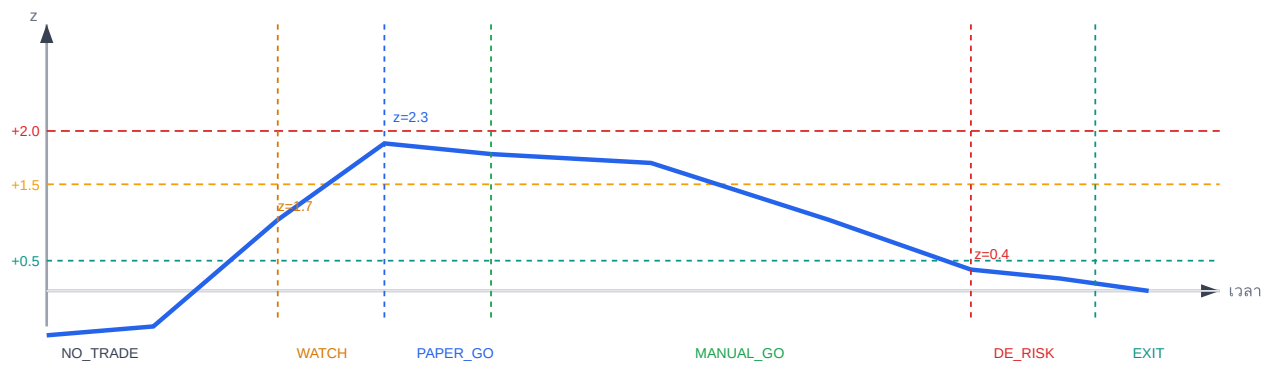
Entry: $z = +2.3$ (Short Bybit ที่แพง, Long Lighter ที่ถูก)

Exit: $z = +0.4$ (spread กลับมาใกล้ค่ากลาง)

กำไร $\approx (2.3 - 0.4) \times \sigma_{\text{spread}} \times \text{position_size} = 1.9\sigma \times \text{ขนาด position}$

เวลาถือ: ~47 นาที ตรงกับ half-life ที่ estimate ไว้ ~45 นาที

การ Visualize Sequence บน z-score Timeline



z-score ขึ้นถึง +2.3 → ผ่านทุก state → กลับมา +0.4 และออกจาก position

11.6 แบบฝึกหัด

โจทย์ 11.1 — Transition ที่ถูกต้อง

ระบบอยู่ใน state MANUAL_GO กับ entry_zscore = +2.5 (Short Bybit, Long Lighter)

ปัจจุบัน z-score = +0.3 และ cointegration ยังคง OK

ถาม: ระบบควร transition ไป state ไหน? เพราะเหตุใด?

► คำตอบ

โจทย์ 11.2 — Emergency Invalidation

ระบบอยู่ใน state MANUAL_GO กำลัง hold position (Short Bybit, Long Lighter)

z-score = +1.8 (ยังไม่ถึง exit threshold) แต่กะทันหัน cointegration test fail (p-value = 0.15)

ถาม: ระบบควรทำอะไร? อธิบาย action และเหตุผล

► คำตอบ

โจทย์ 11.3 — ออกแบบ Anti-Thrashing Logic

ปัญหา: z-score วนอยู่ที่ 1.45–1.55 ทำให้ระบบ transition NO_TRADE → WATCH → NO_TRADE ซ้ำๆ ทุก 2 นาที (เรียกว่า "thrashing")

ถาม: ออกแบบ modification ใน StateMachine เพื่อแก้ปัญหานี้ โดยไม่เปลี่ยน threshold หลัก

► คำตอบ